

Face Entry Pro

Biometric Attendance & Campus Intelligence System

Table of Contents

1. Introduction	2
2. Project Background & Problem Statement	2
• 2.1 Project Background	2
• 2.2 Existing Platforms in Bangladesh & Global Market	3
• 2.3 Problem Statement	3
• 2.4 Motivation (Why This Project is Important)	4
3. Goals & Objectives	5
4. Scope & Deliverables	5
5. Methodology	6
• 5.1 Proposed Solution & Key Features	7
• 5.2 Software Development Approach (SDLC Model)	7
• 5.3 Technologies, Tools & AI Models	7
• 5.4 System Architecture	8
6. Resource Requirements & Budget	8
• 6.1 Hardware Requirements	8
• 6.2 Estimated Budget Table	8
7. Work Plan & Project Timeline	9
• 7.1 Gantt Chart Visualization	10
• 7.2 Team Task Distribution	11
8. Testing & Deployment	11
9. Risks and Challenges	12
10. Conclusion	12
11. References	13

Project Title:

FaceEntryPro : Biometric Attendance & Campus Intelligence System

1. Introduction

Traditional attendance management systems in academic institutions suffer from persistent structural failings, including pervasive proxy marking, human entry errors, data recording delays, and a complete inability to generate actionable insights from the information they collect. Manual paper registers and rudimentary hardware card-swipe frameworks are neither secure nor scalable. Crucially, they offer educators no early-warning capabilities when a student's deteriorating attendance patterns begin to signal academic disengagement and potential dropout risks.

FaceEntryPro directly addresses these operational and security vulnerabilities by delivering an end-to-end, AI-powered Biometric Attendance and Campus Intelligence Platform. At its technological core, the system deploys an advanced Convolutional Neural Network (CNN) trained on student datasets to execute real-time, high-precision face detection and recognition right at campus entry points.

To guarantee operational continuity and strict compliance with modern institutional data-privacy mandates (including **GDPR** frameworks), all biometric recognition inference runs locally on a **Raspberry Pi 4 edge device** utilizing an optimized **TensorFlow Lite** model. This edge-computing approach ensures that the system remains fully operational during local campus network outages and guarantees that sensitive biometric data never traverses a public network or external cloud server.

2. Project Background & Problem Statement

2.1 Project Background

Educational institutions across Bangladesh and globally still rely heavily on manual registers, RFID card-swiping, or basic biometric scanners to monitor student attendance. While digital transformation is sweeping through university management systems, attendance collection remains a bottleneck. Manually logging records or relying on physical cards introduces severe administrative overhead, processing delays, and massive security loopholes.

Furthermore, current frameworks view attendance as a static record-keeping task rather than an analytical asset. The data sits unutilized in local databases until the end of a semester, missing the opportunity to help struggling students when it matters most.

2.2 Existing platforms in Bangladesh:

To position **FaceEntryPro** effectively, we analyzed key market competitors to identify their structural gaps in an academic context:

- **Didit (didit.me)**: A global identity platform featuring advanced **Liveness Detection** to prevent deepfakes and photo-spoofing. However, it relies heavily on cloud APIs for digital onboarding and lacks localized, edge-device integration to manage continuous, physical walkthrough gates or tailgating at campus entry points.
- **EmpMonitor (empmonitor.com)**: A corporate workforce tracking software that uses touchless facial recognition for employee clock-ins and maps productivity analytics. Its infrastructure is built for enterprise desktop monitoring and corporate HR systems, making it unsuited for low-cost micro-controller deployments tailored for university attendance.
- **ScanMax AI (scanmaxai.com)**: Provides highly efficient, specialized commercial facial recognition gates and access terminals. While mechanically robust, it operates strictly as an expensive enterprise security solution, lacking built-in academic predictive analytics or customized student-parent-faculty notification dashboards.

2.3 Problem Statement

The current attendance infrastructures face several critical operational and technical challenges:

Proxy Attendance and Identity Fraud:

Many existing attendance systems can be fooled using another person's card, printed photos, or images on mobile devices. As a result, students may receive attendance without actually being present, reducing the accuracy and reliability of attendance records. .

Network Dependency and Delays:

Most attendance systems rely heavily on an internet connection and central servers. When the network is slow or unavailable, the system may stop working or become very slow, causing long queues and delays at entry points.

No Early Warning for Attendance Issues:

Current systems usually generate attendance reports at the end of the semester. This makes it difficult for teachers to identify students with low attendance early and provide timely support before academic problems become serious.

Security Issues at Entry Points:

Unauthorized individuals may enter campus by following closely behind an authorized person (tailgating). Most traditional attendance systems cannot detect or record these security violations.

Limited Communication with Parents:

Parents often do not have access to real-time attendance information. This lack of transparency makes it difficult for them to monitor their child's campus presence and attendance performance regularly.

2.4 Motivation (Why This Project is Important)

The primary driving force behind this project stems from massive gaps in existing commercial software:

- **Gaps in Existing Solutions:** Platforms like *Didit.me* offer robust facial liveness detection but are engineered as cloud-hosted APIs for remote onboarding, lacking hardware controls for physical security turnstiles. Conversely, workforce trackers like *EmpMonitor.com* provide check-in utilities but are bound to static corporate desktop workstations. Highly specific solutions like *ScanMax AI* provide physical gates but operate as closed corporate ecosystems missing custom academic reporting and predictive student dashboards.
 - **Real-World Impact:** FaceEntryPro bridges these gaps by combining anti-spoofing security, low-cost edge-hardware configurations, and institutional intelligence. It transforms presence tracking from an administrative chore into a life-saving academic tool that preserves institutional data integrity while creating a protective layer around student consistency
-

3. Goals & Objectives

The main goal of FaceEntryPro is to digitize and secure attendance systems in educational institutions.

Specific objectives include:

- Eliminate proxy attendance through biometric verification
- Automate attendance recording process
- Provide real-time attendance tracking
- Generate analytical reports for faculty
- Identify at-risk students early
- Improve transparency and accountability

Secondary Goals:

- Incorporate INT8 quantization pipelines to scale TensorFlow models onto restricted system-on-chip architectures.
 - Incorporate localized SMS/Email alert relays to automate parent-faculty data sync.
-

4. Scope & Deliverables

Project Scope

The system will include the following modules:

- User Authentication System
- Student Registration Module
- Face Recognition Attendance System
- Attendance Management Module
- Analytics and Reporting Dashboard
- Admin and Faculty Panel
- Intelligent Notification System.
- AI-Powered Predictive Analytics
- Security Monitoring Module
- Parent Monitoring System
- Advanced Security Monitoring
- Campus Intelligence Module.

Key Deliverables

- Fully functional web application
 - Face recognition attendance module
 - Database design and implementation
 - System documentation
 - Source code repository
 - Testing and evaluation report
-

5. Methodology

5.1 Proposed Solution

FaceEntryPro proposes an AI-based automated attendance system using facial recognition technology.

Key features include:

- Real-time face recognition-based attendance marking
- Centralized student database management
- Attendance analytics dashboard for faculty
- Automated alerts for low attendance students
- At-risk student prediction based on attendance trends
- Role-based access for students, faculty, and administration
- Parent/guardian monitoring module (optional).
- **Behavioral Violence & Weapon Detection:** Integrating lightweight temporal models (like LSTM or 3D-CNN) alongside face recognition to detect sudden aggressive physical movements, crowding, or suspicious posturing. Once a violence anomaly is flagged, the system immediately captures the faces of the individuals involved in that specific frame bounding-box.
- **Intelligent Post-Event Suspect Search:** If a security breach, vandalism, or violent incident occurs at a specific location, administrators can upload a cropped face snapshot or select a student's profile template. The system will automatically index and scan hours of recorded footage across all campus cameras to isolate every video clip where that specific face appears.

5.2 Software Development Approach

Software Development Lifecycle (SDLC) Model

The platform will be built utilizing the **Agile Development Methodology** mapped into 2-week iterative sprint tracks. This lifecycle structure allows for parallel tracks managing model configuration alongside web endpoint construction, ensuring fluid adaptations based on constant component testing loops.

Phases:

- Requirement analysis
 - System design
 - Module-wise development
 - Testing and validation
 - Deployment and feedback
-

5.3 Technologies & Tools

Frontend:

- HTML, CSS, JavaScript, Bootstrap

Backend:

- Python (Django / Flask)

AI / ML:

- OpenCV
- TensorFlow / Face Recognition Library

Database:

- MySQL

Tools:

- VS Code
- GitHub: For storing all of the code.
- Raspberry Pi 4: For storing data without internet connection.

- Postman
 - MediaPipe Face Detection Framework
 - Greenweb SMS API: For sending notifications or SMS alerts.
 - Vercel: For web hosting.
-

5.4 System Architecture

FaceEntryPro will follow a Three-Tier Architecture:

- Presentation Layer: Web interface for users
- Application Layer: Business logic and AI processing
- Data Layer: Database for storing user and attendance data

6. Resource Requirements & Budget

6.1 Hardware Requirements

- Development laptops/PCs
- Webcam for face recognition testing

6.2 Estimated Budget

Item	Description	Cost (BDT)
Software	Open-source tools	0
Hardware	Camera & testing devices	30,000
Hosting	Cloud/server deployment	5,000
Miscellaneous	Internet & contingency	15,000
Offline tools	Raspberry Pi 4	20,000
SMS tools	Greenweb SMS API	10,000
Total		80,000

7. Work Plan

The project will be completed over a total duration of 14 weeks, following a structured and phased approach.

Week	Milestone	Key Activities
Week 1	Requirement Analysis	Gather requirements, identify stakeholders, analyze existing attendance systems, define functional and non-functional requirements, prepare Software Requirement Specification (SRS).
Week 2-5	System Design	Design system architecture, create use case diagrams, ER diagrams, database schema, UI/UX wireframes, and workflow design.
Week 6	Database Setup	Create database tables, define relationships, implement constraints, configure database server, and perform initial testing.
Week 7-10	Core Development	Develop user authentication, student registration, attendance management, admin panel, faculty dashboard, notification system, and reporting modules.
Week 11	AI Integration	Integrate face recognition model, liveness detection, identity verification, and attendance automation features.

Week 12	Testing	Conduct unit testing, integration testing, system testing, bug fixing, performance evaluation, and security testing.
Week 13-14	Prototype Completion & Documentation	Finalize the prototype, prepare project documentation, create user manuals, generate screenshots and reports, conduct demonstrations, and prepare the final project submission.

After 14 weeks, we still have some tasks to complete.
Like: Project deployment, and maintenance.

7.1 Gantt chart

The Gantt chart below illustrates the project schedule based on weekly progress and task distribution.

Task/Phase	Week 1	Week 2-5	Week 6	Week 7-10	Week 11	Week 12	Week 13-14
Requirement Analysis							
System Design							
Database Setup							
Core Development							
AI Integration							
Testing							

Task/Phase	Week 1	Week 2-5	Week 6	Week 7-10	Week 11	Week 12	Week 13-14
Prototype Completion & Documentation							

7.2 Task Distribution

Team Member and Primary role	Project Proposal Contribution	Technical Contribution
Rakib Hasan Piyas (Team Lead / System Analyst)	Problem identification, literature review, requirement analysis, project planning, proposal writing, scope definition.	System architecture design, use case diagram, workflow design, project integration.
Md Ashraf Siddique (UI/UX & Frontend Developer)	User requirement analysis, interface planning, prototype design documentation.	UI/UX design, frontend development, dashboard creation, responsive interface implementation.
Tahamid Islam Zihad (Backend & Database Developer)	Functional requirement specification, database requirement analysis, module planning.	Backend Development, Database design, API development, authentication system, attendance management and notification modules.
Md Tareq Ahmed (AI/ML & Testing Engineer)	Research on face recognition technologies, feasibility analysis, testing plan preparation.	Face recognition integration, liveness detection, AI model implementation, system testing and validation.

8. Testing & Deployment

Testing

- Unit testing of modules
- Integration testing
- System testing
- User acceptance testing
- Performance testing

Deployment

- Local server deployment for demo
 - Optional cloud deployment
 - Database configuration
 - Security setup and access control
-

9. Risks and Challenges

- High computational cost for face recognition
 - Accuracy issues under poor lighting conditions
 - Data privacy concerns related to biometric data
 - Resistance to adoption from users
 - Network dependency for real-time updates
 - Model training complexity and dataset limitations
-

10. Conclusion

FaceEntryPro provides a modern solution to traditional attendance systems by integrating AI-based face recognition and intelligent analytics. The system improves accuracy, reduces manual workload, and enhances student monitoring capabilities. By implementing this system, institutions can achieve efficient attendance management and gain valuable insights into student engagement and academic risk patterns.

11. References

1. **Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, A., Somani, V., ... & Grundmann, M.** (2019). *MediaPipe: A Framework for Building Perception Pipelines*. In arXiv preprint arXiv:1906.08172. (Used for lightweight, edge-optimized Face Detection algorithms).
2. **Chen, S., Liu, Y., Gao, X., & Han, Z.** (2018). *MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices*. In Biometric Recognition. CCBR 2018. Lecture Notes in Computer Science, vol 10996. Springer, Cham. (Used for high-accuracy embedding generation on constrained System-on-Chips).
3. **Zhang, S., Wang, X., Liu, A., Zhao, C., Wan, J., Sergio, E., & Li, S. Z.** (2020). *A Dataset and Benchmark for Multi-Modal Face Anti-Spoofing*. In IEEE position papers on Presentation Attack Detection (PAD) and MiniFASNet frameworks. (Used for 3D depth-sensing liveness and anti-proxy verification routines).
4. **David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, M., Kaufman, H., ... & Plancher, B.** (2021). *TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems*. In Proceedings of the Machine Learning and Systems (MLSys). (Used for INT8 quantization execution and running inference models locally on Raspberry Pi 4).
5. **Schwaber, K., & Beedle, M.** (2002). *Agile Software Development with Scrum*. Prentice Hall. (Used as the foundational framework to justify the 2-week parallel development tracks of AI optimization and web engineering).
6. **IEEE Standard for Information Technology** (2017). *IEEE Standard for Systems and Software Engineering—Software Life Cycle Processes*. IEEE Std 12207-2017. (Used to benchmark structural SDLC compliance and iterative system testing validation loops).